Light

Character Sets (ASCII/Unicode)

Over **1,100,000+ marks** of exam-aligned content across **67 qualifications**. 4,200+ auto-marked assessments & long-answer practice. No login required.

Assessment Hub | Notes / Textbook | Long-Answer Practice

Assessment Hub
Long-Answer
Notes Index
What you must know
Toolkit
1. Character sets
2. ASCII
3. Unicode
Worked examples
Quick reference
Typical pitfalls

AQA GCSE Computer Science — **Character Sets (8525 Paper 2)**. This topic covers character encoding, including what a character set is, 7-bit ASCII, and Unicode. You must be able to convert between characters and character codes using encoding tables. Exam questions typically ask you to **define** what a character set is, **convert** characters to codes and vice versa, **calculate** character codes using sequential patterns, and **explain** the advantages of Unicode over ASCII. Questions are worth 1—4 marks and often use given encoding tables.

Why character encoding matters

Computers can only process binary — patterns of 0s and 1s. To store and display text (letters, numbers, symbols), every character must be assigned a unique binary code. A **character set** is the agreed list of characters and their corresponding codes that a computer system uses.

Without standardised character encoding, a document created on one computer might display completely different characters when opened on another. Character sets ensure that when you type "Hello", every computer that receives that message displays exactly the same text.

- **ASCII:** An early character set using 7 bits per character, supporting 128 characters — enough for English text
- **Unicode:** A modern character set that can represent characters from virtually all languages and many symbols

Character encoding is fundamental to how computers handle text. Every email you send, every web page you view, every document you create — all rely on character encoding to convert human-readable text into the binary patterns that computers can store, process, and transmit. Understanding this process helps you appreciate how digital communication works at its most basic level.

What you must know

This checklist covers the key exam-visible concepts from the AQA specification. You should be able to explain each point and demonstrate each skill.

- **Understand** what a character set is.
- **Describe** the 7-bit ASCII character encoding method.
- **Describe** the Unicode character encoding method.
- **Convert** characters to character codes using a given encoding table.
- **Convert** character codes to characters using a given encoding table.
- **Understand** that character codes are grouped and run in sequence within encoding tables.
- **Describe** the purpose of Unicode and the advantages of Unicode over ASCII.
- **Know** that Unicode uses the same codes as ASCII up to 127.

> **Specification note.** AQA 8525 assesses character encoding on Paper 2 (Computing concepts). You need to know about 7-bit ASCII and Unicode, but you do NOT need to know about UTF-8, UTF-16, or other specific versions of Unicode. You will always be given an encoding table when asked to convert characters.

Toolkit

Key ASCII codes to remember

You don't need to memorise the entire ASCII table, but knowing these key codes helps you calculate others:

Character	Decimal code	Binary (7-bit)	Notes
A	65	1000001	First uppercase letter
Z	90	1011010	Last uppercase letter
a	97	1100001	First lowercase letter
z	122	1111010	Last lowercase letter
0	48	0110000	Digit zero (not letter O)
9	57	0111001	Digit nine

Character	Decimal code	Binary (7-bit)	Notes
Space	32	0100000	Space character

Sequential pattern rule

Character codes in ASCII are **grouped** and run in **sequence**. This means:

- If A = 65, then B = 66, C = 67, D = 68, and so on
- If a = 97, then b = 98, c = 99, d = 100, and so on
- If 0 = 48, then 1 = 49, 2 = 50, 3 = 51, and so on

Exam technique: If a question gives you the code for 'A' is 65, you can calculate the code for any uppercase letter by counting how many letters it is from A and adding that to 65. For example, 'E' is 4 letters after 'A', so E = 65 + 4 = 69.

Quick calculation formulas

Character to code:

- Uppercase: code = 65 + (position − 1)
- Lowercase: code = 97 + (position − 1)
- Digit: code = 48 + digit value

Code to character:

- If 65—90: uppercase letter at position (code − 64)
- If 97—122: lowercase letter at position (code − 96)
- If 48—57: digit = code − 48

Case conversion

The difference between uppercase and lowercase letters is always exactly 32:

- **Uppercase to lowercase:** add 32 (e.g., A = 65 → a = 97)
- **Lowercase to uppercase:** subtract 32 (e.g., a = 97 → A = 65)

This consistent difference is useful in programming for converting between cases.

Bits and characters

Encoding	Bits per character	Total characters possible
7-bit ASCII	7 bits	2^7 = 128 characters
Extended ASCII	8 bits	2^8 = 256 characters
Unicode	16+ bits	Over 100,000 characters

File size implication: Unicode characters require more bits than ASCII characters, so text files encoded in Unicode are typically larger than the same text encoded in ASCII.

Powers of 2 for character calculations

Remember: n bits can represent 2^n different values:

- 7 bits = 2^7 = 128 values (standard ASCII)
- 8 bits = 2^8 = 256 values (extended ASCII, one byte)
- 16 bits = 2^{16} = 65,536 values (basic Unicode)

1. Character sets and encoding

A **character set** is a collection of characters that a computer can recognise and use. Each character in the set is assigned a unique numeric code. When you press a key on a keyboard, the computer doesn't store the shape of the letter — it stores the numeric code that represents that character.

Character encoding is the process of assigning these numeric codes to characters. Different character encoding systems exist, each with their own set of codes. The two most important for GCSE are ASCII and Unicode.

1.1 What is included in a character set?

A character set typically includes:

- **Letters:** Both uppercase (A—Z) and lowercase (a—z)
- **Digits:** The numbers 0—9
- **Punctuation:** Full stops, commas, question marks, exclamation marks, etc.
- **Special symbols:** @, #, £, $, %, &, *, etc.
- **Control characters:** Non-printable characters like newline, tab, and backspace
- **Space:** The blank character between words

The exact characters included depends on which character set is being used. ASCII was designed primarily for English text, while Unicode was designed to represent characters from all the world's writing systems.

1.2 Why do we need character encoding?

Computers work entirely in binary (0s and 1s). They cannot directly understand letters like "A" or symbols like "@". Character encoding provides a standardised way to convert human-readable text into binary numbers that computers can process, store, and transmit.

Without standardised character encoding:

- Text files created on one computer might display incorrectly on another
- Emails and web pages could show garbled characters
- International communication would be extremely difficult
- Software developers would have to create different versions for different systems

By agreeing on standard character sets, computers worldwide can share text reliably. When you send an email, the characters are converted to their numeric codes, transmitted as binary data, and then converted back to the same characters on the receiving device.

1.3 How character codes work

Each character in a character set has a unique decimal number assigned to it. This decimal number is then stored in binary inside the computer. For example, in ASCII:

- The letter 'A' has decimal code 65, stored as binary 1000001
- The letter 'a' has decimal code 97, stored as binary 1100001
- The digit '5' has decimal code 53, stored as binary 0110101

Notice that uppercase 'A' and lowercase 'a' have different codes — the computer treats them as completely different characters. Similarly, the digit character '5' (code 53) is different from the number 5 used in calculations.

1.4 The encoding and decoding process

When you type text on a computer, a process of encoding and decoding takes place:

Encoding (text → binary):

1. You press a key (e.g., 'H')
2. The keyboard sends a signal to the computer
3. The computer looks up 'H' in the character set and finds code 72
4. The code 72 is stored as binary: 1001000

Decoding (binary → text):

1. The computer reads the binary value 1001000
2. It converts this to decimal: 72
3. It looks up code 72 in the character set and finds 'H'
4. The letter 'H' is displayed on screen

This process happens for every single character in every piece of text you read or write on a computer. The speed of modern processors means this happens almost instantaneously, even for documents with millions of characters.

1.5 Character codes in programming

Understanding character codes is important in programming because you can use them to manipulate text. In Python (and other languages), you can convert between characters and codes:

Python functions for character conversion:

- `ord('A')` returns 65 — converts character to code
- `chr(65)` returns 'A' — converts code to character

This is useful for tasks like sorting text alphabetically, converting case, or validating input.

Worked Example 1 – Defining a character set

Question: Define what is meant by a character set. [2 marks]

Model answer:
A character set is a collection/list of characters **(1)** that a computer can recognise/use, where each character is assigned a unique code/number **(1)**.

Exam tip: For 2-mark definitions, make sure you include two distinct points.

Here: (1) what it contains, and (2) the unique code aspect.

Worked Example 2 — Purpose of character encoding

Question: Explain why character encoding is needed in computer systems. [2 marks]

Model answer:
Computers can only process/store binary **(1)**. Character encoding assigns a unique binary code to each character so that text can be represented/stored/transmitted digitally **(1)**.

Exam tip: Always link back to binary when explaining why encoding is needed. The examiner wants to see you understand that computers fundamentally work in binary.

Worked Example 3 — Characters vs numbers

Question: The digit '7' has ASCII code 55. Explain why this is different from the number 7 used in calculations. [2 marks]

Model answer:
The character '7' is a text symbol with code 55 — it is stored as the binary pattern for 55 **(1)**. The number 7 used in calculations is stored directly as binary 111 (or 0000111) — these are different binary values representing different things **(1)**.

Exam tip: This is a common exam question. Remember: character '7' = code 55 = binary 0110111, but the number 7 for maths = binary 0000111. They are stored differently!

Worked Example 4 — Case sensitivity

Question: In ASCII, 'A' has code 65 and 'a' has code 97. State why uppercase and lowercase letters have different codes. [1 mark]

Model answer:
Each character must have a unique code, and uppercase and lowercase are treated as different characters **(1)**.

> **Exam tip:** The computer cannot "see" that 'A' and 'a' are the same letter — it only sees two different codes (65 and 97).

2. ASCII (7-bit)

ASCII stands for **American Standard Code for Information Interchange**. It was developed in the 1960s and became the standard way to encode text for computers and communication equipment. The original ASCII uses 7 bits to represent each character.

2.1 7-bit ASCII explained

With 7 bits, ASCII can represent 2^7 = **128 different characters** (codes 0—127). These include:

- **Codes 0—31:** Control characters (non-printable) — things like newline, tab, backspace
- **Code 32:** Space character
- **Codes 33—47:** Punctuation and symbols (!, ", #, $, etc.)
- **Codes 48—57:** Digit characters 0—9
- **Codes 58—64:** More punctuation (:, ;, <, =, >, ?, @)
- **Codes 65—90:** Uppercase letters A—Z
- **Codes 91—96:** More symbols ([, \,], ^, _, `)
- **Codes 97—122:** Lowercase letters a—z
- **Codes 123—127:** More symbols and delete character

2.2 The sequential pattern

One of the most important features of ASCII (and one that AQA specifically tests) is that **character codes are grouped and run in sequence**. This means related characters have consecutive codes:

> **Uppercase letters:** A=65, B=66, C=67, D=68, E=69, ... Z=90
> **Lowercase letters:** a=97, b=98, c=99, d=100, e=101, ... z=122
> **Digit characters:** 0=48, 1=49, 2=50, 3=51, 4=52, ... 9=57

This sequential pattern is extremely useful. If you know that 'A' = 65, you can calculate the code for any uppercase letter:

- 'E' is the 5th letter, so E = 65 + 4 = **69**
- 'M' is the 13th letter, so M = 65 + 12 = **77**
- 'Z' is the 26th letter, so Z = 65 + 25 = **90**

Similarly, the difference between uppercase and lowercase is always 32:

- A = 65, a = 97, difference = 32
- B = 66, b = 98, difference = 32
- Z = 90, z = 122, difference = 32

2.3 Using an ASCII table

In the exam, you will be given a character encoding table when you need to convert characters. Here is a partial ASCII table showing the most commonly used characters:

Character	Decimal	Binary (7-bit)
Space	32	0100000

Character	Decimal	Binary (7-bit)
!	33	0100001
0	48	0110000
1	49	0110001
9	57	0111001
A	65	1000001
B	66	1000010
C	67	1000011
Z	90	1011010
a	97	1100001
b	98	1100010
c	99	1100011
z	122	1111010

2.4 Converting characters to codes

Method: Look up the character in the encoding table and read off the decimal code (or binary code if asked).

Example: Convert the word "Hi" to ASCII codes.

- 'H' is the 8th letter. H = 65 + 7 = **72**
- 'i' is the 9th letter. i = 97 + 8 = **105**

Answer: "Hi" = 72, 105

2.5 Converting codes to characters

Method: Find the code in the encoding table and read off the corresponding character. If calculating, determine which group the code belongs to and count from the start of that group.

Example: What character has ASCII code 68?

- 68 is between 65 and 90, so it's an uppercase letter
- 68 − 65 = 3, so it's 3 positions after 'A'
- A, B, C, **D** — the answer is 'D'

2.6 Binary representation of characters

Although exam questions usually ask for decimal codes, it's important to understand that characters are ultimately stored in binary. The process works as follows:

Converting a character to binary:

1. Find the decimal code for the character (e.g., 'H' = 72)
2. Convert the decimal to 7-bit binary (72 = 1001000)
3. This binary pattern is what the computer actually stores

Example: Storing "Hi"

- 'H' = 72 = 1001000
- 'i' = 105 = 1101001
- Stored as: 1001000 1101001

When you save a text file, each character is converted to its binary code and written to storage. When you open the file, the binary codes are read and converted back to characters for display.

2.7 Control characters

ASCII codes 0—31 are reserved for **control characters**. These are non-printable characters that control how text is displayed or processed:

- **Code 10:** Line Feed (LF) — moves to a new line
- **Code 13:** Carriage Return (CR) — moves cursor to start of line
- **Code 9:** Tab — horizontal tab space
- **Code 8:** Backspace — deletes previous character
- **Code 27:** Escape — used for special commands

You don't need to memorise these codes for the exam, but you should understand that not all ASCII codes represent visible characters.

2.8 Extended ASCII

While 7-bit ASCII only uses codes 0—127 (128 characters), most computers store characters in 8-bit bytes. **Extended ASCII** uses the full 8 bits, allowing codes 0—255 (256 characters). The extra 128 characters (codes 128—255) are used for additional symbols.

However, there were many different versions of Extended ASCII, each adding different characters in the 128—255 range. This caused compatibility problems — a document created on one system might display incorrectly on another. This inconsistency was one of the reasons Unicode was developed.

> **Specification note.** AQA focuses on 7-bit ASCII (128 characters). You should be aware that Extended ASCII exists and uses 8 bits (256 characters), but detailed knowledge of specific Extended ASCII character sets is not required.

2.9 Limitations of ASCII

While ASCII was revolutionary for its time, it has significant limitations:

- **Only 128 characters:** 7 bits limits the total to 128 unique characters
- **English-centric:** Designed for American English — no accented characters (é, ñ, ü)
- **No other alphabets:** Cannot represent Greek, Cyrillic, Arabic, Chinese, Japanese, or other scripts
- **Limited symbols:** Many mathematical and scientific symbols are missing
- **No emojis:** Modern communication symbols cannot be represented

These limitations led to the development of Unicode, which we cover in the next section.

2.10 Calculating file sizes with ASCII

Because ASCII uses a fixed number of bits per character, you can calculate the storage required for text:

File size calculation:
File size (bits) = Number of characters × bits per character
File size (bytes) = Number of characters × bytes per character
For 7-bit ASCII stored in bytes: File size = Number of characters × 1 byte
Example: A document with 5000 characters in ASCII = 5000 bytes = 5 kB (using decimal prefixes)

Remember that AQA uses decimal prefixes: 1 kB = 1000 bytes, 1 MB = 1000 kB.

Worked Example 5 — Character to code conversion

Question: Using the ASCII table where A = 65, convert the character 'K' to its ASCII decimal code. [1 mark]

Model answer:
'K' is the 11th letter of the alphabet.
K = 65 + 10 = **75**

Exam tip: Count carefully! 'A' is position 1, so 'K' (position 11) is 10 positions after A, not 11.

Worked Example 6 — Code to character conversion

Question: Using ASCII where a = 97, what character has code 109? [1 mark]

Model answer:
109 − 97 = 12
12 letters after 'a' = **'m'**

Exam tip: For lowercase, start from 97. Count: a(97), b(98), c(99)... or just subtract 97 and count.

Worked Example 7 — Converting a word

Question: Convert the word "Cat" to ASCII decimal codes. You are given: A = 65, a = 97. [2 marks]

Model answer:
'C' = 65 + 2 = 67 **(1)**
'a' = 97
't' = 97 + 19 = 116

Answer: **67, 97, 116 (1)**

Exam tip: Watch for mixed case! 'C' is uppercase (starts at 65), but 'a' and 't' are lowercase (starts at 97).

Worked Example 8 — Calculating number of characters

Question: ASCII uses 7 bits per character. Calculate how many different characters can be represented using 7-bit ASCII. [1 mark]

Model answer:
2^7 = **128 characters**

Exam tip: Use the formula: number of possible values = 2^n where n is the number of bits.

3. Unicode

Unicode is a modern character encoding standard designed to represent characters from all the world's writing systems. While ASCII was sufficient for basic English text, the global nature of computing demanded a more comprehensive solution.

3.1 The purpose of Unicode

Unicode was created to solve the limitations of ASCII by providing:

- **Universal coverage:** Characters from virtually all written languages (Latin, Greek, Cyrillic, Arabic, Chinese, Japanese, Korean, etc.)
- **Mathematical symbols:** Comprehensive mathematical and scientific notation
- **Technical symbols:** Musical notation, currency symbols, arrows, shapes
- **Emojis:** The pictorial symbols used in modern messaging
- **Historical scripts:** Ancient languages and historical alphabets

Unicode currently defines codes for over **140,000 characters** from more than 150 scripts, with room to grow.

3.2 How Unicode relates to ASCII

An important design decision in Unicode is that it maintains **backward compatibility with ASCII**:

Key fact: Unicode uses the same codes as ASCII for characters 0—127.

- In ASCII: A = 65
- In Unicode: A = 65

This means any text encoded in ASCII can be read correctly as Unicode, making migration between systems easier.

However, Unicode extends far beyond ASCII's 128 characters. Unicode can represent characters with codes up to 1,114,112 (though not all are currently assigned).

3.3 Bits and storage in Unicode

Because Unicode needs to represent so many more characters, it typically uses more bits per character than ASCII:

- **ASCII:** 7 bits per character (stored in 1 byte)
- **Unicode:** Commonly 16 bits or more per character

This means Unicode text files are typically larger than ASCII files containing the same text. For example, the word "Hello" would require:

- **ASCII:** 5 × 7 = 35 bits (stored in 5 bytes)
- **Unicode (16-bit):** 5 × 16 = 80 bits (stored in 10 bytes)

> **Specification note.** AQA does not require you to know about UTF-8, UTF-16, or other specific Unicode encoding formats. You just need to understand that Unicode uses more bits than ASCII and can represent more characters.

3.4 Advantages of Unicode over ASCII

This is a common exam question. The main advantages are:

More characters

Unicode can represent over 140,000 characters compared to ASCII's 128. This allows for a far greater range of characters to be encoded.

International alphabets

Unicode supports characters from virtually all the world's writing systems, including Chinese, Japanese, Arabic, Greek, and Cyrillic scripts.

Accented characters

Characters with accents (é, ñ, ü, ç) can be represented directly, essential for languages like French, Spanish, and German.

Special symbols

Mathematical symbols (Σ, ∞, ≠), currency symbols (€, ¥, £), and emojis can all be represented in Unicode.

3.5 Disadvantages of Unicode

While Unicode is far more capable than ASCII, it does have some drawbacks:

- **Larger file sizes:** More bits per character means larger text files
- **More memory required:** Processing Unicode text uses more memory than ASCII
- **Slower processing:** More data to handle can mean slightly slower operations

- **Compatibility:** Very old systems may not fully support Unicode

However, for modern computing, the benefits of Unicode far outweigh these disadvantages. Most systems now use Unicode as the default encoding.

3.6 Calculating file sizes with Unicode

Because Unicode uses more bits per character, files encoded in Unicode are larger than the same text in ASCII. Here's how to calculate and compare:

Comparing ASCII and Unicode file sizes:

Consider the text "Hello World" (11 characters including space):

- **ASCII (7-bit, stored in 1 byte each):** 11 × 1 = 11 bytes
- **Unicode (16-bit):** 11 × 2 = 22 bytes

The Unicode version is exactly twice the size for simple English text.

This size difference becomes significant for large documents. A 100,000-character document would be 100 kB in ASCII but 200 kB in 16-bit Unicode. However, modern storage is inexpensive, and the benefits of international character support usually outweigh the storage cost.

3.7 Real-world applications

Unicode is essential for:

- **Websites:** Web pages can display content in any language using Unicode
- **Social media:** Users can post in their native languages and use emojis
- **Email:** International email communication with proper character display
- **Software:** Applications can be localised for different regions without code changes
- **Documents:** Word processors can handle multilingual documents seamlessly
- **Operating systems:** Windows, macOS, Linux, iOS, and Android all use Unicode internally

3.8 The importance of standardisation

The Unicode Consortium is the organisation responsible for maintaining and developing the Unicode standard. They ensure that:

- Every character has one and only one unique code
- New characters (like new emojis) are added in a controlled, standardised way
- Historical scripts and rare characters are preserved
- All computers worldwide can communicate text reliably

Without this standardisation, international digital communication would be far more difficult. Unicode has become one of the most important standards in computing, enabling the truly global internet we have today.

3.9 Exam scenarios involving character encoding

AQA exam questions on character encoding often use real-world scenarios. Here are common types of questions you might encounter:

Scenario 1: International website

"A company wants to create a website that can be viewed in English, Chinese, and Arabic. Explain why Unicode would be more suitable than ASCII for this website."

Key points: Unicode can represent characters from multiple alphabets/scripts; ASCII is limited to English characters only.

Scenario 2: File storage

"A document contains 10,000 characters. Calculate the file size in ASCII and Unicode (16-bit). Explain the difference."
Key points: ASCII = 10,000 bytes; Unicode = 20,000 bytes. Unicode requires more bits per character.

Scenario 3: Character conversion

"Using the given table where A=65, convert the password 'Cat123' to ASCII codes."
Key points: Mixed case (C=67, a=97, t=116) and digits (1=49, 2=50, 3=51). Watch for capital vs lowercase!

Scenario 4: Sequential codes

"In ASCII, 'M' has code 77. State the code for 'P'."
Key points: P is 3 letters after M, so P = 77 + 3 = 80. Use the sequential pattern.

3.10 Linking to other topics

Character encoding connects to several other areas of the GCSE Computer Science course:

- **Data representation:** Characters are just another form of data that must be represented in binary, alongside numbers, images, and sound
- **File sizes:** The choice of character encoding affects how much storage text requires — Unicode files are larger than ASCII files
- **Compression:** Understanding character codes helps with Huffman coding — you can compare the bits saved against ASCII encoding
- **Programming:** String operations in Python use character codes internally; functions like `ord()` and `chr()` convert between characters and codes
- **Networks:** When data is transmitted over networks, text is sent as binary character codes that must be decoded at the receiving end
- **Input/Output:** When you type on a keyboard, the key press is converted to a character code; when text is displayed on screen, codes are converted back to visible characters

Understanding character encoding helps you see how computers handle one of the most fundamental types of data — text. This knowledge is essential for understanding how digital communication works and why international websites and applications need to use Unicode rather than the older ASCII standard.

Worked Example 9 — Explaining Unicode's purpose

Question: Describe the purpose of Unicode. [2 marks]

Model answer:
Unicode allows representation of characters from many different languages/alphabets **(1)** and special symbols such as emojis and mathematical

notation that cannot be represented in ASCII **(1)**.

Exam tip: Focus on what Unicode can do that ASCII cannot — international characters and special symbols.

Worked Example 10 – Advantages of Unicode

Question: State two advantages of Unicode over ASCII. [2 marks]

Model answer:
1. Unicode can represent characters from many different alphabets/languages (e.g., Chinese, Arabic) **(1)**
2. Unicode can represent a far greater range of characters (over 140,000 vs 128) **(1)**

Exam tip: Other valid answers include: accented characters, mathematical symbols, emojis. Make each point distinct — don't just rephrase the same advantage twice.

Worked Example 11 – ASCII and Unicode compatibility

Question: Explain how ASCII and Unicode are related. [2 marks]

Model answer:
Unicode uses the same character codes as ASCII for codes 0—127 **(1)**. This means ASCII text can be read/interpreted correctly as Unicode (backward compatibility) **(1)**.

Exam tip: The key number to remember is 127 — Unicode matches ASCII exactly up to this code.

Worked Example 12 – Comparing file sizes

Question: A text file contains 1000 characters. Explain why this file would be larger if encoded using Unicode rather than ASCII. [2 marks]

Model answer:

Unicode uses more bits per character than ASCII (e.g., 16 bits vs 7 bits) **(1)**. Therefore, each character requires more storage space, making the overall file larger **(1)**.

Exam tip: You can include specific numbers: ASCII uses 7 bits (1 byte), Unicode commonly uses 16 bits (2 bytes) per character.

Worked examples summary

This page contains 12 worked examples covering all the key skills for character sets. Use this table to find specific types of questions for revision.

Example	Topic	Question type	Marks
1	Defining a character set	Define	2
2	Purpose of character encoding	Explain	2
3	Characters vs numbers	Explain	2
4	Case sensitivity	State	1
5	Character to code conversion	Calculate	1
6	Code to character conversion	Calculate	1
7	Converting a word	Calculate	2
8	Number of characters	Calculate	1
9	Purpose of Unicode	Describe	2
10	Advantages of Unicode	State	2
11	ASCII and Unicode compatibility	Explain	2
12	Comparing file sizes	Explain	2

Quick reference

Character set comparison

Feature	7-bit ASCII	Unicode
Bits per character	7 bits	16+ bits
Total characters	128	Over 140,000
English letters	Yes	Yes

Feature	7-bit ASCII	Unicode
Digits	Yes	Yes
Basic punctuation	Yes	Yes
Accented characters	No	Yes
Non-Latin alphabets	No	Yes
Emojis	No	Yes
File size	Smaller	Larger
Codes 0—127	Full set	Same as ASCII

Key ASCII codes

Group	Characters	Code range	Starting point
Digits	0—9	48—57	'0' = 48
Uppercase	A—Z	65—90	'A' = 65
Lowercase	a—z	97—122	'a' = 97
Space	(space)	32	—

Conversion methods

Conversion	Method
Character → Code	Find starting code for that group (65 for uppercase, 97 for lowercase, 48 for digits). Count position in group. Add position to starting code.
Code → Character	Determine which group the code belongs to. Subtract the starting code. Count that many positions from the first character.
Upper → Lower	Add 32 to the code (e.g., A=65 → a=97)
Lower → Upper	Subtract 32 from the code (e.g., a=97 → A=65)

Key definitions for the exam

Term	Definition (mark-scheme style)
Character set	A collection of characters that a computer can recognise, where each character has a unique code.

Term	Definition (mark-scheme style)
Character encoding	The process of assigning unique numeric codes to characters.
ASCII	American Standard Code for Information Interchange — a 7-bit character encoding supporting 128 characters.
Unicode	A character encoding standard that can represent characters from virtually all writing systems worldwide.
Character code	The unique numeric value assigned to a character in a character set.

Typical pitfalls

Pitfall 1 — Confusing character '5' with number 5.
Wrong: "The character '5' is stored as binary 101."
Right: The character '5' has ASCII code 53, stored as binary 0110101. The number 5 for calculations is stored as binary 101.

Pitfall 2 — Forgetting that uppercase and lowercase are different.
Wrong: "'A' and 'a' have the same code."
Right: 'A' = 65 and 'a' = 97. They are completely different characters with different codes (32 apart).

Pitfall 3 — Off-by-one counting errors.
Wrong: "'C' is 3 letters from A, so C = 65 + 3 = 68."
Right: 'C' is 2 positions after 'A' (A→B→C), so C = 65 + 2 = 67.

Pitfall 4 — Saying Unicode "can represent all characters".
Wrong: "Unicode represents every possible character."
Right: Unicode represents characters from virtually all/most writing systems. Be precise — don't overclaim.

Pitfall 5 — Forgetting backward compatibility.
Wrong: "ASCII and Unicode use completely different codes."
Right: Unicode uses the same codes as ASCII for characters 0—127.

Pitfall 6 — Not knowing the sequential pattern.
Wrong: "You need to memorise every ASCII code."
Right: Codes run in sequence: if A=65, then B=66, C=67, etc. You only need to know starting points.

Pitfall 7 — Confusing bits with bytes.
Wrong: "ASCII uses 7 bytes per character."
Right: ASCII uses 7 bits per character (stored in 1 byte). Unicode uses 16 or more bits.

Pitfall 8 — Vague advantages of Unicode.
Wrong: "Unicode is better than ASCII."
Right: Give specific advantages: more characters, international alphabets, accented characters, emojis/symbols.

Pitfall 9 — Forgetting the 2^n formula.
Wrong: "7 bits can represent 7 characters."
Right: n bits can represent 2^n values. So 7 bits = 2^7 = 128 characters.

Pitfall 10 — Mentioning UTF-8 or UTF-16.
Wrong: Discussing specific Unicode encoding formats in detail.
Right: AQA spec says these are NOT required. Just say "Unicode" — don't complicate it.

Assessment Hub | Long-Answer | Notes Index

Williams Physics Education — over **1,100,000+ marks** of exam-aligned content and **4,200+ auto-marked assessments** across 67 qualifications at GCSE, A-Level, AP & Professional Qualifications. Free to use, no login required.

About & Contact • Privacy • Terms • Accessibility • Copyright • DMCA • YouTube

WPE is free for everyone. If our resources help you or your students, you can support the project here.

© 2026 Williams Physics Education. All rights reserved.